

Securing Tenant Data in the Marvin Knowledge Base Service

Overview

The Knowledge Base (KB) service is a Retrieval-Augmented Generation (RAG) system built on **Amazon Bedrock Knowledge Bases**. Customers upload documents, which are chunked, embedded, and stored so they can be queried in natural language.

Because multiple customers' documents can be embedded into a **shared** Bedrock Knowledge Base, every layer of the system identity, storage, and retrieval is scoped by a tenant identity of `CustomerId + OrganizationId`, with an additional `CollectionId` scope inside a tenant. This page describes how those boundaries are enforced.

1. What customer data we hold and where it lives

Data	Store	Tenant scoping
Raw uploaded documents (files)	Amazon S3 — <code>mc- <env>-knowledge- base-docs</code>	Object key prefixed <code>customer_id/organ ization_id/collec tion_id/document_ id/name</code>
Multimodal / supplemental extracted data	Amazon S3 — <code>mc- <env>-knowledge- base-multimedia</code>	Same prefixing scheme
Document / collection / label metadata	PostgreSQL (RDS)	Rows carry <code>customer_id</code> , <code>organization_id</code> , <code>collection_id</code>
Vector embeddings of document chunks	Amazon S3 Vectors , fronted by a Bedrock Knowledge Base	Each embedding carries <code>customer_id</code> , <code>organization_id</code> , <code>collection_id</code> , <code>document_id</code> as metadata

2. How we store the data securely

- **Encryption at rest:** Both S3 document buckets use server-side encryption by default (**SSE-S3** / **AES256** , with S3 Bucket Keys enabled).
- **Encryption in transit:** Both buckets have a bucket policy (**DenyNonTLS**) that denies all S3 actions when **aws:SecureTransport = false** .
- **No public access:** Full **PublicAccessBlockConfiguration** enabled on both buckets.
- **Least-privilege IAM:** The service Lambda role is granted S3 object access only on the **mc-<env>-knowledge-base-docs** bucket.
- **Per-tenant key prefixing:** Every object is written under a **customer_id/organization_id/collection_id/...** key prefix, and the exact key is recorded in the database.
- **Tenant metadata on every embedding:** On ingestion, a Bedrock metadata sidecar file (**<s3_key>.metadata.json**) is written alongside each document; Bedrock attaches these attributes to **every** chunk/embedding created from it (see Section 4).

3. How each request is authorized and mapped to a tenant

Every request is authenticated by **Amazon Cognito** before it reaches application code:

- **REST APIs** are protected by a **Cognito User Pool authorizer** on API Gateway, which validates the JWT and injects its claims.
- **The KB assistant (invoke) path** runs on the Bedrock AgentCore runtime with a **CUSTOM_JWT** authorizer; the handler additionally re-verifies the bearer token's signature against the Cognito pool's JWKS.

Once authenticated, the caller's tenant identity is resolved as follows:

1. We take **only the Cognito sub claim** (the **CognitoSub**) from the validated token.
2. We use it to look up the user in the SQL **Users** table, matching on the **CognitoSub** column.
3. From that row we read **CustomerId** , **OrganizationId** , and **UserId** and assemble them into a request-scoped context object (**day2_context**).

```
1 # users_repo_sql.py - identity resolution
2 cognito_sub = query_param.get("sub")
3 user_obj = (
4     self.session_ro.query(Users)
5     .filter(and_(Users.CognitoSub == cognito_sub))
6     .options(load_only(Users.CustomerId, Users.UserId,
7         Users.OrganizationId))
8     .first()
9 )
10 if user_obj is None:
11     raise ValueError("User not found")
```

Key points:

- The tenant identifiers are **never taken from the request body or client input**, they are derived server-side from the authenticated `CognitoSub` via the trusted `Users` table.
- If no `Users` row matches the `CognitoSub`, identity resolution **fails closed** with `ValueError("User not found")`, and the request is rejected before any data access occurs.
- The resolved `CustomerId` and `OrganizationId` are stamped on every embedding (Section 4) and applied as a filter on every query (Section 5).

4. Tenant identity stamped on every embedding (write path)

During ingestion, the service writes a Bedrock metadata sidecar file next to each document. Bedrock applies these `metadataAttributes` to every chunk/embedding it generates:

```

1 # upload_metadata.py
2 metadata_key = f"{s3_key}.metadata.json"
3 metadata_content = {
4     "metadataAttributes": {
5         "customer_id": customer_id,
6         "organization_id": organization_id,
7         "collection_id": collection_id,
8         "document_id": document_id,
9     }
10 }
11

```

As a result, **no embedding exists in the vector store without a `customer_id`, `organization_id`, and `collection_id` stamped on it.**

5. How retrieval respects tenant boundaries (read path)

When a user queries the knowledge base:

1. `CustomerId` and `OrganizationId` are read from the authenticated `day2_context` (never from the query itself) and passed into the retrieval tool. If either is absent from the context, the request fails before any retrieval runs.
2. The target collection is loaded **scoped to the tenant**; if it does not belong to the caller, the request is rejected with *"Collection not found or access denied."*
3. The vector search runs with a **mandatory metadata filter** requiring **all** of `customer_id`, `organization_id`, and `collection_id` to match:

```

1 # knowledge_base_tools.py – every retrieval is filtered
2 "filter": {
3     "andAll": [
4         {"equals": {"key": "customer_id", "value": customer_id}},
5         {"equals": {"key": "organization_id", "value":
6             organization_id}},

```

```
6         {"equals": {"key": "collection_id", "value": collection_id}},
7     ]
8 }
9
```

Because the filter uses `andAll`, a chunk is only eligible if it matches every condition. This metadata filter is the primary control that keeps tenants separated inside a shared Bedrock Knowledge Base.

Summary of controls

- **Authentication:** every request validated by Cognito.
 - **Trusted identity:** tenant IDs derived server-side from the Cognito `sub` via the `Users` table — never from client input.
 - **Fail-closed identity:** unknown users rejected before any data access.
 - **Write-time tagging:** `customer_id` + `organization_id` + `collection_id` on every embedding.
 - **Read-time filtering:** every vector search filtered by all three.
 - **Storage hardening:** SSE-S3 at rest, TLS-only, public access blocked, least-privilege IAM, per-tenant key prefixing.
-